

任务 1：架构风格选择与论证

项目：CampusHub 校园互助服务平台

本任务以团队最终架构设计文档为前提，最终方案确定为：

前后端分离 + 模块化单体 + 后端 MVC/分层架构 + 局部事件机制预留。

对应技术栈为：

- 前端：Vue 3、Vite、TypeScript、Element Plus、Pinia、Axios
- 后端：Spring Boot 3、MyBatis-Plus、JWT、Redis、Redisson
- 数据库：PostgreSQL
- 交互方式：RESTful API + 标准 JSON 响应，实时通知场景预留 WebSocket

1. 项目背景与架构约束

CampusHub 面向校园互助场景，核心功能包括用户注册登录、实名认证、任务发布与浏览、抢单、订单流转、双向评价、信用分、站内消息、订单聊天、系统公告、校园论坛、后台管理与统计等。

结合最终架构文档，本项目架构选择面临以下约束：

1. 团队规模有限：团队共有 4 名成员，课程项目更适合选择学习成本可控、协作边界清楚的架构。
2. 项目周期有限：Phase 2 之后仍需完成编码、测试和交付，不宜把大量时间投入分布式基础设施。
3. 功能模块较多：系统包含 user、task、order、review、message、forum、common、bootstrap 等模块，需要清晰的模块边界。
4. 一致性要求集中：抢单、订单取消、订单完成、信用分变更等链路需要明确的事务边界和状态控制。
5. 并发目标中等：当前非功能需求主要是关键页面 2 秒内加载、至少支持 100 名用户同时在线，单体应用配合 PostgreSQL、Redis、Redisson 和索引优化即可满足。
6. 扩展需求明确：后续可能扩展实时通知、推荐、图片/文件服务、小程序端或搜索能力，因此架构需要保留演进空间。

2. 候选架构方案

本次分析选取 3 种候选方案：

- 方案 A：前后端分离 + 模块化单体 + MVC/分层架构
- 方案 B：微服务架构
- 方案 C：事件驱动架构作为主架构

说明：MVC/分层架构主要解决单个后端应用内部职责划分问题；模块化单体解决业务域组织问题；微服务解决大型系统的独立部署和独立扩展问题；事件驱动主要解决模块或服务之间的异步协作问题。它们不是完全互斥的，本项目最终采用的是以方案 A 为主体，并局部吸收事件机制的组合方案。

3. 候选方案对比表

对比维度	方案 A：前后端分离 + 模块化单体 + MVC/分层	方案 B：微服务架构	方案 C：事件驱动主架构
开发复杂度	中低。前端 Vue 3 + TypeScript 按页面、布局、组件、状态、请求层组织；后端 Spring Boot 3 按 Controller、Service、Mapper、DTO/Entity 分层，并以 Maven 多模块划分业务边界。	高。需要拆分用户服务、任务服务、订单服务、消息服务、论坛服务等，并额外引入网关、服务发现、配置中心、链路追踪等治理能力。	中高。需要完整设计事件、消费者、消息可靠性、幂等、重试、死信和最终一致性；对课程项目首版过重。
可维护性	高。user、task、order、review、message、forum、common、bootstrap 模块边界清楚，代码集中部署，便于调试和重构。	理论上高，但前提是服务边界成熟、团队具备 DevOps 能力；否则容易形成分布式单体。	对异步扩展友好，但业务流程分散在事件消费者中，排查问题和测试链路会更复杂。
可扩展性	中高。模块化单体可先保证交付，未来可优先拆分消息、文件、搜索、推荐等边界清晰模块。WebSocket 和消息队列也可后续接入。	高。服务可以独立部署和扩容，适合大规模用户和多团队协作。	高。新增通知、统计、推荐等下游处理时，只需订阅事件，生产者改动较少。
性能	能满足当前需求。单体内部调用无网络开销；PostgreSQL 负责事务与查询，Redis/Redisson 支撑缓存和抢单并发控制。	服务可独立扩容，但跨服务调用会增加网络延迟和故障点；当前规模下收益不明显。	异步削峰能力强，适合消息通知、日志、统计；但核心订单链路仍需要同步事务控制。
团队学习成本	低到中。技术栈与团队已有设计一致，Spring Boot 分层和 Vue 组件化易于理解。	高。需要学习服务治理、分布式事务、容器部署、监控告警等内容。	中高。需要理解事件建模、消息一致性、幂等处理和故障补偿。
10 周内	高。可以把时间集中在核心业务、接口联调和测试上。	低。基础设施与服务拆分会消耗大量时间，核	中。若只局部使用可行；若作为

对比维度	方案 A：前后端分离 + 模块化单体 + MVC/分层	方案 B：微服务架构	方案 C：事件驱动主架构
可行性		心业务完整性风险较高。	主架构，设计和调试成本偏高。
与最终架构文档匹配度	最高。最终文档采用 Vue 3 + Vite + TypeScript 前端、Spring Boot 3 + MyBatis-Plus 后端、PostgreSQL + Redis/Redisson，并以后端多模块组织业务。	中。可作为未来演进路线，但不是当前最终架构。	中。最终架构仅把事件用于任务发布、订单创建/完成、消息通知、信用分更新等局部解耦场景。

4. 最终选择的架构形态

最终架构形态如下：



5. 架构选择结论

团队最终选择：

前后端分离 + 模块化单体 + MVC/分层架构，局部预留事件机制。

具体含义是：

1. 前端采用 Vue 3 + Vite + TypeScript 的单页应用架构，按页面视图、布局、组件、状态管理和请求层组织。
2. 后端采用 Spring Boot 3 Maven 多模块模块化单体，统一部署，但内部按业务域拆分模块。
3. 后端每个业务模块内部采用 Controller、Service、Mapper、Entity、DTO 的分层结构。
4. 数据库使用 PostgreSQL，Redis 用于缓存、会话控制和并发辅助，Redisson 用于抢单场景的锁控制。
5. 订单状态、抢单、取消、完成等核心链路使用同步调用和事务控制。
6. 消息通知、信用分变更、任务发布、订单完成等非核心链路通过应用内事件进行解耦，并为未来接入消息队列保留空间。

6. 选择理由

1. 与最终需求范围匹配：CampusHub 功能多但规模仍属于校园级应用，模块化单体足以覆盖任务、订单、论坛、消息和后台管理。
2. 与开发周期匹配：10 周课程项目需要快速进入实现阶段，微服务或完整事件驱动会明显挤占开发和测试时间。
3. 与团队能力匹配：Vue 3、Spring Boot 3、MyBatis-Plus、PostgreSQL、Redis 是更容易掌控的技术组合。
4. 与一致性要求匹配：抢单和订单状态流转适合放在同一个后端应用和数据库事务边界内处理。
5. 与性能目标匹配：单体内部调用成本低，PostgreSQL 索引、Redis 缓存和 Redisson 锁可以支撑当前并发目标。
6. 与未来演进匹配：模块化边界清晰，后续可逐步拆分消息、文件、搜索、推荐等独立能力。

7. 为什么不选择微服务作为当前主架构

微服务架构具备独立部署、独立扩容和技术栈灵活等优点，但本项目当前不选择它作为主架构，主要原因如下：

1. 当前业务规模不足以抵消微服务带来的治理成本。
2. 团队人数较少，拆成多个服务会增加联调、测试和部署难度。
3. 用户、任务、订单、信用分、消息之间存在较多一致性关联，微服务会引入分布式事务或最终一致性问题。
4. 当前非功能目标是 100 名用户同时在线，模块化单体配合缓存和索引即可满足。
5. 一开始使用微服务容易把课程项目变成基础设施项目，影响核心功能交付。

因此，微服务更适合作为未来用户量扩大、团队规模扩大后的演进方向。

8. 为什么不选择事件驱动作为当前主架构

事件驱动适合解耦和异步处理，但不适合作为 CampusHub 首版的主架构：

1. 抢单、订单确认、取消、完成等操作需要同步返回明确结果。
2. 完整事件驱动需要处理消息丢失、重复消费、乱序、重试和幂等，复杂度高。
3. 课程项目当前没有必要引入完整消息队列体系。
4. 事件链路过多会让测试和问题排查变困难。

最终方案保留事件机制，但只用于局部解耦，例如：

- TaskPublishedEvent：任务发布后触发消息或统计更新
- OrderCreatedEvent：订单创建后触发通知
- OrderCompletedEvent：订单完成后触发评价入口、信用分变更和消息通知
- UserCreditEvent：信用分变更后触发等级展示或通知

9. 任务 1 结论

综合开发复杂度、可维护性、可扩展性、性能、团队学习成本和 10 周内可行性，团队确认最终架构为：

前后端分离 + 模块化单体 + 后端 MVC/分层架构 + 局部事件机制预留。

该方案既能支持最终架构文档中定义的前端页面、后端模块、接口、数据库表和并发控制设计，又能避免微服务和完整事件驱动在课程项目阶段带来的过度复杂度。